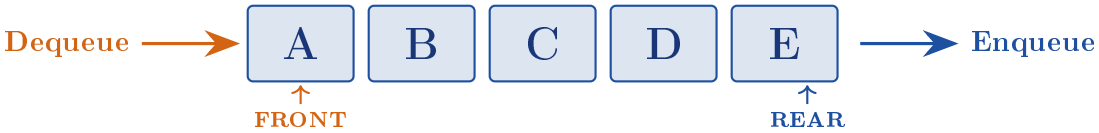
Queue Data Structure

A Complete Beginner-to-Advanced Reference in C++

Linear Queue · Circular Queue · Linked-List Queue · Algorithms & Complexity



"First In, First Out — the fairest discipline in computing."

- ▶ What is a data structure?
- ▶ Memory, arrays, pointers
- ▶ What is a Queue (FIFO)?
- ▶ Queue ADT operations
- ▶ Linear Queue (full code)
- ▶ False Overflow (with diagrams)
- ▶ Circular Queue (full code)
- ▶ Linked-List Queue (full code)
- ▶ Reverse and rotate algorithms
- ▶ Overflow / underflow handling
- ▶ Bug explanations & fixes
- ▶ Time & Space complexity tables

All code is corrected, optimised, and memory-safe. Every concept is explained from scratch with diagrams.

Contents

1 Background: What You Need to Know First **3**
- 1.1 What Is a Data Structure? . . . 3
- 1.2 How Computer Memory Works . . . 3
- 1.3 Arrays in C++ . . . 4
- 1.4 Pointers in C++ . . . 4
- 1.5 Dynamic Memory Allocation (`new` and `delete`) . . . 4
- 1.6 Structs and Classes in C++ . . . 5

2 Introduction to the Queue **5**
- 2.1 What Is a Queue? . . . 6
- 2.2 FIFO vs LIFO — Two Fundamental Orders . . . 6
- 2.3 The Two Core Pointers: `front` and `rear` . . . 6
- 2.4 Real-World Applications . . . 7

3 The Queue ADT (Abstract Data Type) **7**
- 3.1 Queue ADT Operations . . . 8
- 3.2 Overflow and Underflow . . . 8

4 Types of Queues **8**

5 Linear Queue — Array-Based Implementation **9**
- 5.1 Conceptual Design . . . 9
- 5.2 Enqueue: Inserting an Element . . . 9
- 5.3 Dequeue: Removing an Element . . . 9
- 5.4 The False Overflow Problem . . . 10
- 5.5 Complete Linear Queue C++ Code . . . 11
  - 5.5.1 Critical Bug Fix 1: Off-by-One in Compaction . . . 14
  - 5.5.2 Critical Bug Fix 2: Upper Bound in `reverseUpToK` . . . 14
  - 5.5.3 Step-by-Step Trace: Compaction During Enqueue . . . 14

6 Circular Queue — Array-Based Implementation **15**
- 6.1 The Core Idea: The Modulo Ring . . . 15
- 6.2 Diagram: The Circular Array Ring . . . 16
- 6.3 Full and Empty Distinction . . . 16
- 6.4 Complete Circular Queue C++ Code . . . 16
- 6.5 Step-by-Step Trace: Wraparound During Enqueue . . . 19

7 Linked-List Queue **19**
- 7.1 Why Use a Linked List? . . . 19
- 7.2 Node Structure . . . 19
- 7.3 Memory Layout Diagram . . . 20
- 7.4 Complete Linked-List Queue C++ Code . . . 20
- 7.5 All Critical Bug Fixes Explained in Detail . . . 23
  - 7.5.1 Bug 1: Missing `return` in `CreateQueue()` . . . 23
  - 7.5.2 Bug 2: `rear_` Not Reset on Last Dequeue . . . 23

7.5.3 Bug 3: `count_` Not Incremented for First Insertion 24

8 Algorithm Deep Dives **24**
8.1 Algorithm 1 — Reverse Queue Using a Stack 24
8.2 Algorithm 2 — Reverse In-Place (Two-Pointer) 25
8.3 Algorithm 3 — Left-Rotate by k Positions 25

9 Time and Space Complexity Analysis **25**
9.1 Operation Complexities 26
9.2 Understanding Amortised $O(1)$ 26
9.3 When to Choose Which Implementation 27

10 Common Mistakes Cheat-Sheet **27**

11 Summary and Quick Reference **28**
11.1 Implementation Comparison Cheat-Sheet 28
11.2 Classic Queue-Based Interview Problems 28

1. Background: What You Need to Know First

This section is for complete beginners. It covers just enough C++ and computer memory concepts to understand everything that follows. If you are already comfortable with pointers, arrays, and classes, skip to Section 2.

1.1. What Is a Data Structure?

When you write a program, you need to **store** and **organise** data so it can be retrieved and processed efficiently. A **data structure** is a specific way of organising data in the computer's memory.

Real-World Analogy

Imagine two ways of storing 500 books:
(a) Piled randomly on the floor.
(b) Sorted on shelves by genre, then alphabetically by author.

Finding a specific book in (a) requires checking every book — slow! In (b), you go directly to the right shelf and letter — fast. A data structure is the organisational scheme that makes (b) possible for data.

Different problems require different structures:

- Process tasks in arrival order ⇒ **Queue**
- Undo the last action ⇒ **Stack**
- Fast key-based lookup ⇒ **Hash Table**
- Sorted data, fast search ⇒ **Binary Search Tree**

1.2. How Computer Memory Works

Your computer's RAM (Random Access Memory) is like a very long row of numbered boxes, called **memory cells**. Each box holds exactly one **byte** (8 bits). Every box has a unique **address** (a number).

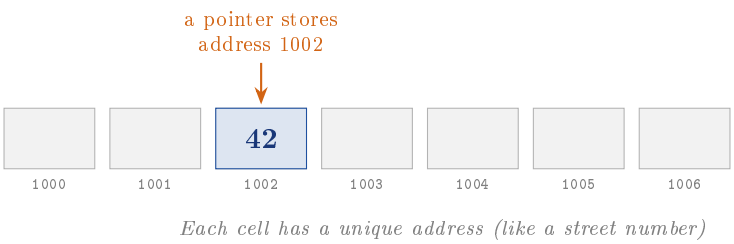


Figure 1: Computer memory: each byte has a unique numeric address. A *pointer* is a variable that stores such an address.

1.3. Arrays in C++

An **array** is a contiguous (physically adjacent) sequence of memory cells, all of the same type. Declaring `int arr[5]` reserves five consecutive `int`-sized blocks.

```
int arr[5];          // 5 consecutive int cells, indices 0
    to 4
arr[0] = 10;         // first cell   (index 0, not 1!)
arr[1] = 20;
arr[4] = 50;         // last cell    (index 4, not 5!)
// arr[5] is ILLEGAL -- out of bounds, undefined behaviour
```

Listing 1: Array basics in C++

Note

Array indices in C++ **always start at 0**. An array of size n has valid indices $0, 1, 2, \ldots, n-1$. Accessing `arr[n]` goes past the end and causes undefined behaviour (crash or silent corruption).

1.4. Pointers in C++

A **pointer** is a variable whose value is a memory **address**. Pointers let us refer to a location in memory indirectly.

```
int x = 42;
int* ptr = &x;    // ptr holds the ADDRESS of x
                  // the & operator means "address of"

cout << x;        // prints 42       (the value stored in x)
cout << ptr;      // prints 1002     (the address of x, e.g.
     0x7ffd...)
cout << *ptr;     // prints 42       (* means "value at this
     address")

*ptr = 100;       // changes x to 100 by writing through
    the pointer
cout << x;        // prints 100
```

Listing 2: Pointer basics

1.5. Dynamic Memory Allocation (`new` and `delete`)

Normal arrays need their size known at compile time. With `new`, you allocate memory at **runtime**:

```
int n;
cin >> n;
int* arr = new int[n];    // allocate n ints on the heap (
    runtime)

// ... use arr ...
```

```
delete[] arr;              // MUST release memory when
    finished!
arr = nullptr;             // safety: prevent dangling
    pointer
```

Listing 3: Dynamic allocation

Common Mistake

If you use `new` but never call `delete[]`, the memory stays allocated forever — this is a **memory leak**. In long-running programs, leaks eventually exhaust all available memory, causing crashes. **Every `new` must be paired with exactly one `delete`.**

1.6. Structs and Classes in C++

A **struct** (or **class**) groups related data and functions under one name:

```
struct Node {
    int   data;     // the value stored in this node
    Node* next;     // pointer to the next Node (can be
        nullptr)
};

class Queue {
public:              // members are accessible from outside
    the class
    int* arr;        // pointer to the storage array
    int  front;      // index of the first element
    int  rear;       // index of the last element

    void enqueue(int val) { /* insert */ }
    int  dequeue()        { /* remove and return */ }
};
```

Listing 4: Struct and class basics

Now we have all the background we need. Let us study the Queue.

2. Introduction to the Queue

2.1. What Is a Queue?

Definition

A **Queue** is a linear data structure that follows the **FIFO** principle: **First In, First Out**. The element that was inserted earliest is always removed first. New elements are added at one end (the **rear**), and elements are removed from the opposite end (the **front**).

Real-World Analogy

Think of a line of customers waiting at a bank. The first person to join the line gets served first. Nobody can cut in. New customers join at the back. This is exactly the queue data structure.

2.2. FIFO vs LIFO — Two Fundamental Orders

Acronym	Meaning	Structure	Example
FIFO	First In, First Out	Queue	Bank queue, print spooler
LIFO	Last In, First Out	Stack	Undo history, function calls

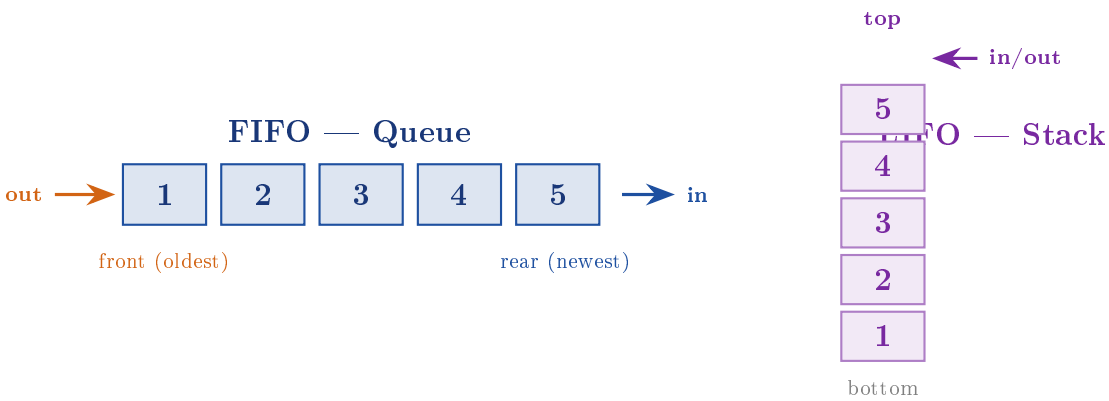


Figure 2: FIFO (Queue): removal happens at the opposite end from insertion. LIFO (Stack): both insertion and removal happen at the same end (top).

2.3. The Two Core Pointers: `front` and `rear`

Every queue tracks two positions:

- `front` — index/pointer to the element that will be removed *next* (the oldest element).
- `rear` — index/pointer to the most recently inserted element. New elements go just after this position.

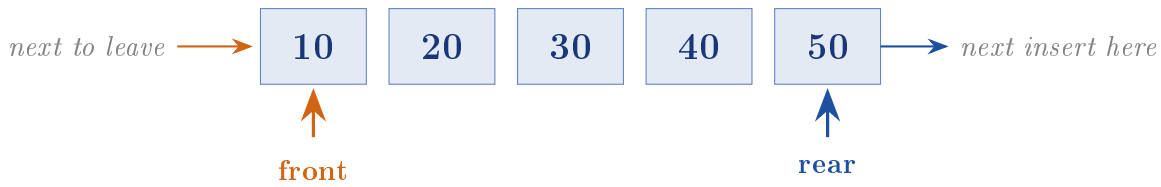


Figure 3: A queue of five elements: `front` marks the exit side; `rear` marks the last inserted element.

2.4. Real-World Applications

Queues appear everywhere in computing:

- **CPU Scheduling:** The OS keeps a "ready queue" of processes waiting for CPU time.
- **Keyboard Buffer:** Keystrokes are stored in a queue; the application reads them in order.
- **Print Spooler:** Multiple documents sent to one printer are queued and printed one at a time.
- **Network Routers:** Packets waiting to be forwarded are buffered in queues.
- **Breadth-First Search (BFS):** Uses a queue to explore graph nodes level by level.
- **Call Centres:** Incoming calls are placed in a queue and answered in arrival order.
- **Web Servers:** Incoming HTTP requests are queued when the server is busy.

3. The Queue ADT (Abstract Data Type)

Definition

An **Abstract Data Type (ADT)** specifies *what operations* a data structure supports and *what they do*, without specifying *how* they are implemented internally. It is a contract between the user and the implementer.

3.1. Queue ADT Operations

Operation	Description	Return Type
`enqueue(x)`	Insert element `x` at the rear of the queue.	void
`dequeue()`	Remove the front element and return its value.	element
`peek() / front()`	Return the front element *without* removing it.	element
`isEmpty()`	Is the queue currently empty?	bool
`isFull()`	Is the queue at capacity? (bounded implementations only)	bool
`size()`	How many elements are in the queue right now?	int

3.2. Overflow and Underflow

Overflow

Calling `enqueue` on a *full* queue. No space to insert. Must be detected and handled gracefully.

Underflow

Calling `dequeue` or `peek` on an *empty* queue. No element to return. Must be detected and handled gracefully.

4. Types of Queues

- **Linear Queue (Simple Queue):** The most basic form. Uses an array with `front` and `rear` indices. Suffers from the "false overflow" problem discussed in Section 5.
- **Circular Queue:** Fixes false overflow permanently by treating the array as a ring. The modulo operator wraps pointers around. This is the standard, production-quality array queue.
- **Double-Ended Queue (Deque):** Insertion and deletion allowed at *both* ends. C++ STL provides `std::deque`.
- **Priority Queue:** Each element has a priority value; elements are served in priority order. Implemented with a binary heap. C++ STL provides `std::priority_queue`.

- **Linked-List Queue:** Uses dynamically allocated nodes instead of a fixed array. Grows and shrinks on demand.

5. Linear Queue — Array-Based Implementation

5.1. Conceptual Design

We allocate an array of fixed size `capacity`. Two integer variables track the queue boundaries:

- `front`: index of the oldest element (next to be dequeued).
- `rear`: index of the newest element (most recently enqueued).

Initial state: `front = 0`, `rear = -1` (no elements yet).
Empty condition: `front > rear` (equivalently, `front == rear + 1`).
Full condition: `rear == capacity - 1` AND `front == 0`.

5.2. Enqueue: Inserting an Element

1. Check for overflow. If the queue is genuinely full, report error and stop.
2. Increment `rear` by 1.
3. Store the new value at `arr[rear]`.

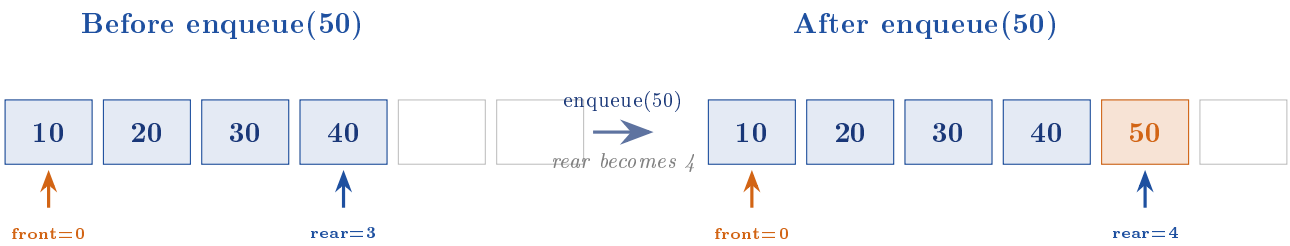


Figure 4: Enqueue: `rear` increments by one and the new value is placed at the new `rear` position.

5.3. Dequeue: Removing an Element

1. Check for underflow. If empty, report error and stop.
2. Save the value at `arr[front]`.
3. Increment `front` by 1 (the old front slot is now "freed").
4. Return the saved value.

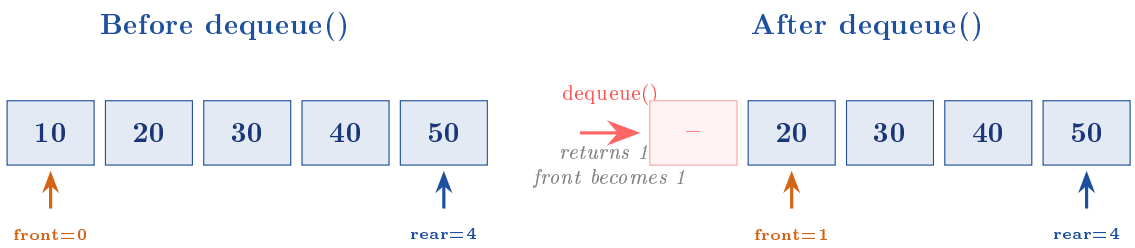


Figure 5: Dequeue: value 10 is returned; `front` advances. Slot 0 is now permanently wasted in a na
ive linear queue.

5.4. The False Overflow Problem

After repeated enqueue-dequeue cycles, `front` creeps rightward, leaving "dead" (freed) slots at the beginning. Eventually `rear` hits `capacity - 1` and the queue reports overflow — even though there may be many free slots at the start.

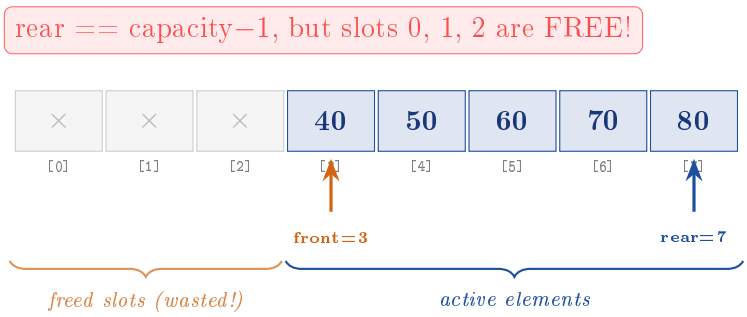


Figure 6: False Overflow: a na
ive linear queue reports "full" even though indices 0–2 are vacant.

Linear queue fix: When false overflow is detected (`rear == capacity-1` and `front > 0`), shift all live elements back to index 0. This is correct but costs $O(n)$ time. The **circular queue** (Section 6) solves this permanently in $O(1)$.

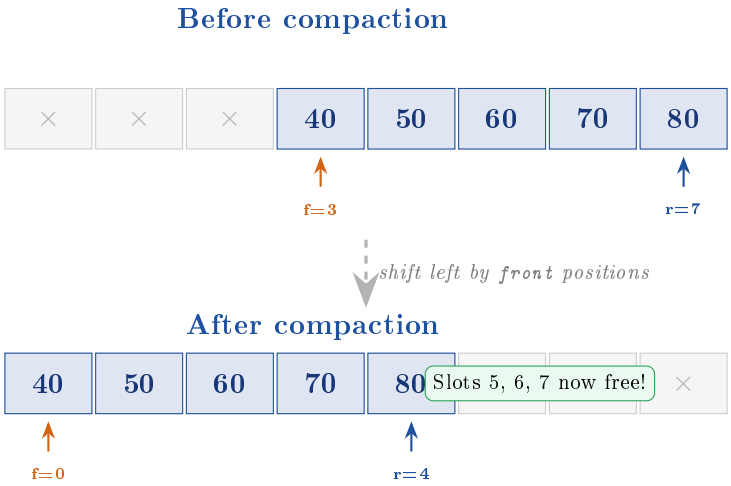


Figure 7: Compaction recovers the wasted space at the cost of $O(n)$ time.

5.5. Complete Linear Queue C++ Code

```cpp
#include <iostream>
#include <stack>
using namespace std;

class Queue {
public:
    int* arr;        // pointer to heap-allocated storage array
    int  capacity;   // maximum number of elements
    int  front;      // index of the first (oldest) element
    int  rear;       // index of the last (newest) element

    // ?? Constructor ??????????????????????????????????????
    // capacity = size, front starts at 0, rear starts at -1
    // (rear == -1 means queue is empty: rear < front)
    Queue(int size) : capacity(size), front(0), rear(-1) {
        arr = new int[size];
    }

    // ?? Destructor ??????????????????????????????????????
    // Called automatically when the Queue object goes out of scope.
    // Without this, the array on the heap is NEVER freed (memory leak).
    ~Queue() {
        delete[] arr;
    }

    // ?? Capacity checks ??????????????????????????????????
    // True full: array is genuinely at capacity.
    bool isFull() const {
        return (rear == capacity - 1) && (front == 0);
    }
    // True empty: front has passed rear; no elements.
    bool isEmpty() const {
        return (front > rear);
    }
    // Number of live elements.
    int size() const {
        return rear - front + 1;
    }

    // ?? Enqueue ??????????????????????????????????????????
    void enqueue(int val) {
        if (isFull()) {
```

```
            cout << "Queue is FULL!\n";
            return;
        }
        // Detect false overflow and compact the array
        if (rear == capacity - 1 && front > 0) {
            int n = size();                     // count of live elements
            for (int i = 0; i < n; i++)
                arr[i] = arr[front + i];        // shift elements left
            front = 0;
            rear  = n - 1;     // BUG FIX: was 'n+1'; correct is 'n-1'
        }
        arr[++rear] = val;                      // place the new element
    }

    // ?? Dequeue ??????????????????????????????????????????????
    // Returns -1 as a sentinel if the queue is empty.
    int dequeue() {
        if (isEmpty()) return -1;
        int val = arr[front];
        arr[front++] = -1;       // mark old slot as freed (visual aid)
        return val;
    }

    // ?? Peek ????????????????????????????????????????????????
    int peek() const {
        if (isEmpty()) return -1;
        return arr[front];
    }

    // ?? Algorithm 1: Reverse entire queue using a stack ?????
    // Idea: push all elements onto a LIFO stack; pop them back.
    // The stack's reversal property gives us a reversed queue.
    // Time: O(n)   Space: O(n)
    void reverseQueue() {
        stack<int> s;
        int n = size();
        for (int i = 0; i < n; i++)
            s.push(dequeue());
        while (!s.empty()) {
            enqueue(s.top());
            s.pop();
```

```cpp
        }
    }

    // ?? Algorithm 2: Reverse entire queue in-place ??????????
    // Idea: elements are contiguous in arr[front..rear].
    // Swap the outermost pair and work inward (two-pointer).
    // Time: O(n)   Space: O(1)
    void reverseInPlace() {
        int lo = front;
        int hi = rear;
        while (lo < hi) {
            swap(arr[lo], arr[hi]);
            lo++;
            hi--;
        }
    }

    // ?? Algorithm 3: Reverse first k elements in-place ??????
    // Only the sub-array arr[front .. front+k-1] is reversed.
    // Precondition: 1 <= k <= size()
    // Time: O(k)   Space: O(1)
    void reverseUpToK(int k) {
        int lo = front;
        int hi = front + k - 1;     // BUG FIX: was (k-1)-front
        while (lo < hi) {
            swap(arr[lo], arr[hi]);
            lo++;
            hi--;
        }
    }

    // ?? Display ??????????????????????????????????????????
    void show() const {
        if (isEmpty()) { cout << "(empty)\n"; return; }
        for (int i = front; i <= rear; i++)
            cout << arr[i] << (i < rear ? " -> " : "\n");
    }
};

// ?? Driver ??????????????????????????????????????????????????
int main() {
    Queue q(10);
    for (int v : {1, 2, 3, 4, 5, 6, 7, 8, 9})
        q.enqueue(v);
```

```

    cout << "Original queue:\n";
    q.show();

    cout << "\nAfter reverseUpToK(3):\n";
    q.reverseUpToK(3);
    q.show();

    cout << "\nAfter full reverseInPlace():\n";
    q.reverseInPlace();
    q.show();

    cout << "\nDequeuing all: ";
    while (!q.isEmpty())
        cout << q.dequeue() << " ";
    cout << "\n";
    return 0;
}
```

Listing 5: Linear Queue — Array-Based, Fully Corrected Implementation

5.5.1. Critical Bug Fix 1: Off-by-One in Compaction

After shifting n elements to positions $0, 1, \ldots, n-1$, the last element sits at index $n-1$. So `rear` must be set to $n-1$:

```
rear = n - 1;     // CORRECT: last shifted element is at
    index n-1
// NOT:
rear = n;         // WRONG:  leaves a gap at n-1
rear = totalEle; // WRONG:  same error with a different
    variable name
```

5.5.2. Critical Bug Fix 2: Upper Bound in `reverseUpToK`

The first k elements occupy indices `front` through `front + k - 1`:

```
int hi = front + k - 1;    // CORRECT
// NOT:
int hi = (k - 1) - front; // WRONG: negative when front >
    k-1!
```

If `front = 3` and `k = 2`, the wrong formula gives `hi = (2-1) - 3 = -2`. Writing to `arr[-2]` is undefined behaviour — a silent, potentially catastrophic bug.

5.5.3. Step-by-Step Trace: Compaction During Enqueue

State: capacity = 6, array = `[-, -, 30, 40, 50, 60]`, `front = 2`, `rear = 5`.
Call: `enqueue(70)`.

Step-by-Step Trace

Step 1	`isFull()`? `rear==5==cap-1` yes; `front==0`? No (front=2). Not truly full.
Step 2	False overflow: `rear==cap-1 && front>0`. True. Compact.
Step 3	`n = size() = 5 - 2 + 1 = 4`.
Step 4	Shift: `arr[0]=30, arr[1]=40, arr[2]=50, arr[3]=60`.
Step 5	Reset: `front=0, rear=3`.
Step 6	`arr[++rear] = 70` ⇒ `arr[4]=70, rear=4`.
Result	`[30,40,50,60,70,-]`, `front=0`, `rear=4`. ✓

6. Circular Queue — Array-Based Implementation

6.1. The Core Idea: The Modulo Ring

Instead of shifting elements, we use a single arithmetic trick: **modulo** (`%`).

$$\text{rear} \leftarrow (\text{rear} + 1) \bmod \text{capacity}$$

When `rear` = `capacity - 1`, the formula gives (capacity-1 + 1) mod capacity = 0. The rear pointer wraps around to index 0 automatically — free slots that were "freed" by earlier dequeues are reused. The array behaves as a logical ring.

Key Insight

There is no longer any difference between "the front of the array" and "the back of the array". Every slot is equally available for reuse. This makes both `enqueue` and `dequeue` $O(1)$ in *all* cases, with zero wasted space.

6.2. Diagram: The Circular Array Ring

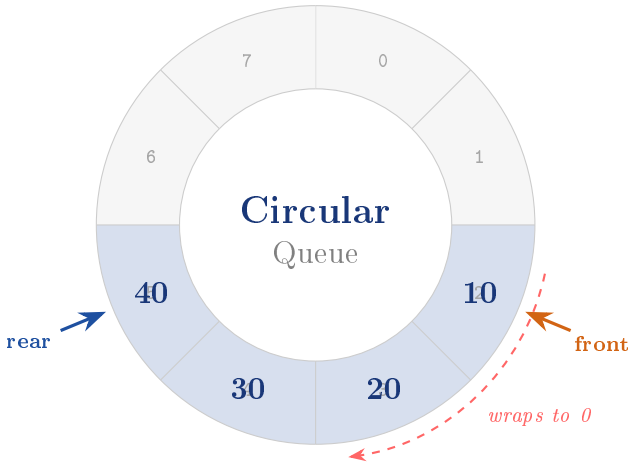


Figure 8: Circular Queue as a ring: when `rear` reaches index 7, the next enqueue wraps it back to 0 (if free).

6.3. Full and Empty Distinction

Without a counter, both the full state and the empty state can produce `front == (rear+1)%capacity`, which is ambiguous. We resolve this by maintaining a `count` variable:

- **Empty:** `count == 0`
- **Full:** `count == capacity`

6.4. Complete Circular Queue C++ Code

```cpp
#include <iostream>
using namespace std;

class CircularQueue {
public:
    int* arr;
    int  front, rear;
    int  capacity, count;

    // ?? Constructor ??????????????????????????????????????????
    explicit CircularQueue(int cap = 100)
        : capacity(cap), front(0), rear(-1), count(0) {
        arr = new int[cap];
    }

    // ?? Destructor ???????????????????????????????????????????
    ~CircularQueue() { delete[] arr; }

```

```cpp
    // ?? Checks ??????????????????????????????????????????????
    bool isFull()  const { return count == capacity; }
    bool isEmpty() const { return count == 0; }
    int  size()    const { return count; }

    // ?? Enqueue: O(1) always ????????????????????????????????
    // rear wraps around using modulo -- the circular trick.
    void enqueue(int val) {
        if (isFull()) { cout << "Queue is full!\n"; return; }
        rear = (rear + 1) % capacity;   // wrap if needed
        arr[rear] = val;
        count++;
    }

    // ?? Dequeue: O(1) always ????????????????????????????????
    void dequeue() {
        if (isEmpty()) { cout << "Queue is empty!\n"; return; }
        arr[front] = -1;                        // mark freed (optional)
        front = (front + 1) % capacity;         // wrap if needed
        count--;
    }

    // ?? Peek ????????????????????????????????????????????????
    int peek() const {
        if (isEmpty()) return -1;
        return arr[front];
    }

    // ?? Algorithm: Left-rotate by k positions ?????????????????
    // Move the first k elements to the rear one at a time.
    // Normalising k avoids redundant full rotations.
    // Time: O(k)   Space: O(1)
    void rotateByK(int k) {
        if (isEmpty() || k <= 0) return;
        k %= count;     // a full rotation is an identity operation
        for (int i = 0; i < k; i++) {
            enqueue(arr[front]);
            dequeue();
        }
```

```cpp
    }

    // ?? Display ??????????????????????????????????????????
    // MUST use count as loop bound -- not 'while(temp<= rear)'!
    // When front > rear (wrapped state), the naive condition fails.
    void show() const {
        if (isEmpty()) { cout << "(empty)\n"; return; }
        int idx = front;
        for (int i = 0; i < count; i++) {
            cout << arr[idx];
            if (i < count - 1) cout << " -> ";
            idx = (idx + 1) % capacity;    // correct wraparound
        }
        cout << "\n";
    }

    void totalElements() const {
        cout << "Total elements: " << count << "\n";
    }
};

// ?? Driver ??????????????????????????????????????????????
int main() {
    CircularQueue q(8);

    cout << "Enqueuing 1 to 8:\n";
    for (int v : {1,2,3,4,5,6,7,8})
        q.enqueue(v);
    q.totalElements();
    q.show();

    cout << "\nDequeue 3 elements:\n";
    q.dequeue(); q.dequeue(); q.dequeue();
    q.show();

    cout << "\nEnqueue 9, 10, 11 (reuse freed slots):\n";
    q.enqueue(9); q.enqueue(10); q.enqueue(11);
    q.show();

    cout << "\nrotateByK(2):\n";
    q.rotateByK(2);
    q.show();
    return 0;
}
```

Listing 6: Circular Queue — Full-Featured, Memory-Safe Implementation

6.5. Step-by-Step Trace: Wraparound During Enqueue

Setup: capacity = 5. Enqueue 5 elements, dequeue 2, then enqueue 2 more.

Step-by-Step Trace

Operation	front	rear	count	Array (logical)
initial	0	-1	0	[, , , ,]
enqueue(10)	0	0	1	[10, , , ,]
enqueue(20)	0	1	2	[10, 20, , ,]
enqueue(30)	0	2	3	[10, 20, 30, ,]
enqueue(40)	0	3	4	[10, 20, 30, 40,]
enqueue(50)	0	4	5	[10, 20, 30, 40, 50]
dequeue() $\rightarrow$ 10	1	4	4	[$--$, 20, 30, 40, 50]
dequeue() $\rightarrow$ 20	2	4	3	[$--$, $--$, 30, 40, 50]
enqueue(60)	2	**0**	4	[60, $--$, 30, 40, 50]
enqueue(70)	2	**1**	5	[60, 70, 30, 40, 50]

rear wraps: $(4+1)\%5 = 0$, then $(0+1)\%5 = 1$. Slots 0 and 1 are reused. Queue is full again — **no shifting, no wasted space.** ✓

7. Linked-List Queue

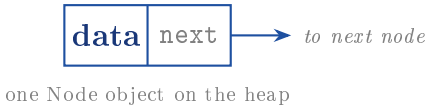
7.1. Why Use a Linked List?

Array-based queues need a fixed size at construction time. If you do not know how many elements you will need, you must over-allocate (wasting memory) or risk overflow. A **linked-list queue** allocates exactly one node per element on the heap, so the queue grows and shrinks on demand with no wasted space.

Property	Array Queue	Linked-List Queue
Capacity	Fixed at construction	Unlimited (heap only)
False overflow	Possible	Impossible
Memory overhead	1 int per slot	1 int + 1 pointer per node
Cache efficiency	Excellent (contiguous)	Lower (scattered on heap)
Enqueue	$O(1)$ / $O(n)$ worst	$O(1)$ always
Dequeue	$O(1)$	$O(1)$

7.2. Node Structure

Each element in a linked-list queue is stored in a **node**: a small heap object that holds the data value and a pointer to the next node.



7.3. Memory Layout Diagram

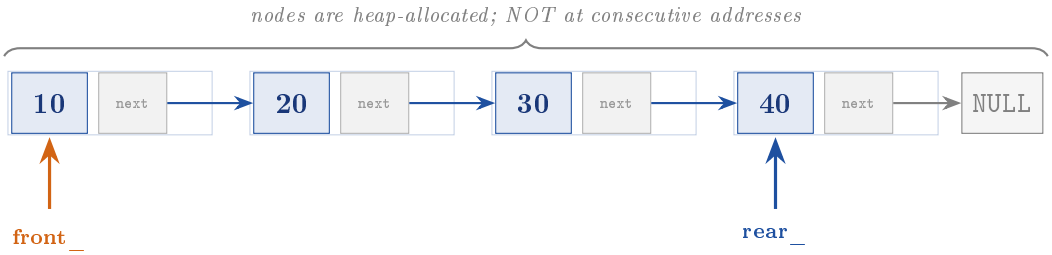


Figure 9: Linked-list queue memory layout. `front_` points to the oldest node; `rear_` points to the newest. The `next` pointers chain them logically.

7.4. Complete Linked-List Queue C++ Code

```cpp
#include <iostream>
using namespace std;

// ?? Node ????????????????????????????????????????????????????????????
struct Node {
    int   data;
    Node* next;
    explicit Node(int val) : data(val), next(nullptr) {}
};

// ?? Queue ????????????????????????????????????????????????????????????
class Queue {
private:
    Node* front_;     // oldest element (exit side)
    Node* rear_;      // newest element (entry side)
    int   count_;

public:
    Queue() : front_(nullptr), rear_(nullptr), count_(0) {}

    // ?? Destructor ??????????????????????????????????????????????
    // Without this, every Node remaining in the queue leaks!
    ~Queue() {
        while (!isEmpty())
            dequeue();
    }
```

```cpp

    // ?? Checks ??????????????????????????????????????????????
    bool isEmpty() const { return front_ == nullptr; }
    int  size()    const { return count_; }

    // ?? Enqueue: O(1) ??????????????????????????????????????
    void enqueue(int val) {
        Node* newNode = new Node(val);
        if (rear_ == nullptr) {          // queue was empty
            front_ = rear_ = newNode;
        } else {
            rear_->next = newNode;       // link after current rear
            rear_       = newNode;       // advance rear pointer
        }
        count_++;    // BUG FIX: original missed this for first insert
    }

    // ?? Dequeue: O(1) ??????????????????????????????????????
    void dequeue() {
        if (isEmpty()) return;
        Node* temp = front_;
        front_     = front_->next;
        if (front_ == nullptr)           // BUG FIX: reset rear_ too!
            rear_ = nullptr;
        temp->next = nullptr;            // isolate the node
        delete temp;                     // free the memory
        count_--;
    }

    // ?? Peek: O(1) ??????????????????????????????????????????
    int peek() const {
        if (isEmpty()) return -1;
        return front_->data;
    }

    // ?? Search: O(n) ????????????????????????????????????????
    // Returns 1-based position, or -1 if not found.
    int search(int x) const {
        Node* curr = front_;
```

```cpp
        for (int pos = 1; curr != nullptr; pos++, curr = curr->next)
            if (curr->data == x) return pos;
        return -1;
    }

    // ?? Display: O(n) ??????????????????????????????????????
    void show() const {
        if (isEmpty()) { cout << "(empty)\n"; return; }
        for (Node* curr = front_; curr != nullptr; curr = curr->next)
            cout << curr->data << (curr->next ? " -> " : " -> NULL\n");
    }

    void totalElements() const {
        cout << "Size: " << count_ << "\n";
    }
};

// ?? Factory function ??????????????????????????????????????
// BUG FIX: original had no return statement -- undefined behaviour!
Queue* createQueue() {
    return new Queue();     // constructor initialises everything
}

// ?? Driver ????????????????????????????????????????????????
int main() {
    Queue* q = createQueue();
    q->enqueue(1);
    q->enqueue(2);
    q->enqueue(3);
    q->enqueue(4);

    cout << "Queue contents:\n";
    q->show();
    q->totalElements();

    int pos = q->search(3);
    if (pos != -1)
        cout << "Value 3 is at position " << pos << "\n";

    cout << "\nDequeuing all: ";
    while (!q->isEmpty()) {
        cout << q->peek() << " ";
        q->dequeue();
```

```
    }
    cout << "\n";

    delete q;     // destructor frees all remaining nodes
    return 0;
}
```

Listing 7: Linked-List Queue — Complete, Memory-Safe Implementation

7.5. All Critical Bug Fixes Explained in Detail

7.5.1. Bug 1: Missing `return` in `CreateQueue()`

Common Mistake

```
Queue* CreateQueue(){
    Queue* q = new Queue();
    q->front = NULL;
    q->rear  = NULL;
    q->count = 0;
    // NO RETURN STATEMENT!
}
```

In C++, a non-void function that reaches the end without a `return` statement has **undefined behaviour**. The caller receives a garbage pointer. Dereferencing it crashes the program.

Fix: `return new Queue();` — the constructor already initialises all fields.

7.5.2. Bug 2: `rear_` Not Reset on Last Dequeue

Common Mistake

```
// After last dequeue:
front_ = front_->next;     // front_ becomes nullptr
// BUT rear_ still points to the DELETED node --
   dangling pointer!

// The next enqueue does:
rear_->next = newNode;     // writing to freed memory
   -- CRASH
```

Fix:

```
front_ = front_->next;
if (front_ == nullptr)     // queue is now empty
    rear_ = nullptr;       // reset rear_ as well
```

7.5.3. Bug 3: count_ Not Incremented for First Insertion

Common Mistake

```
if(front == NULL){
    front = rear = newNode;
    return;                 // count++ is SKIPPED for
        the first element
}
rear->next = newNode;
rear = newNode;
count++;                    // only reached for
    subsequent insertions
```

After the first element is inserted, `count` is still 0. The `searchEle` loop runs 0 iterations and never finds anything. **Fix:** move `count++` to after both branches.

8. Algorithm Deep Dives

8.1. Algorithm 1 — Reverse Queue Using a Stack

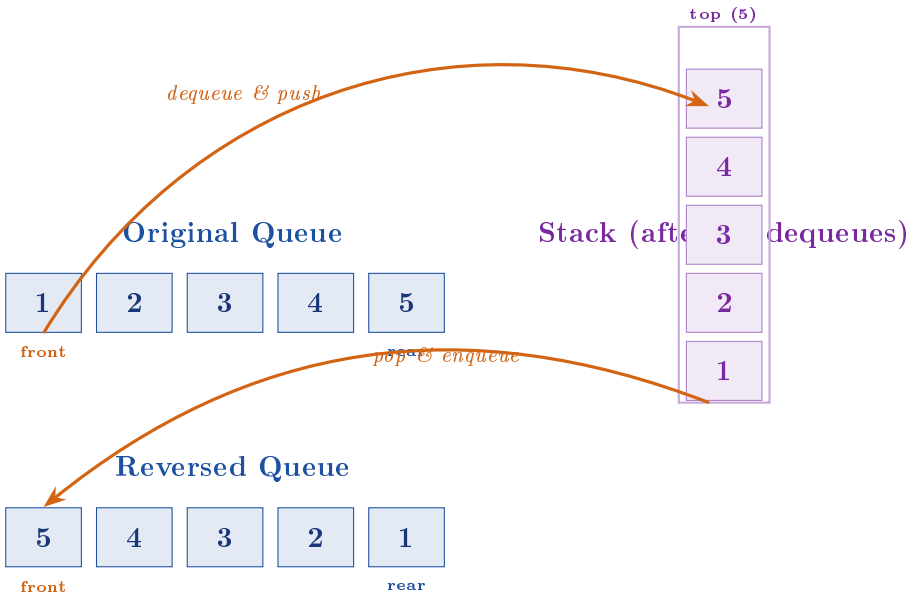


Figure 10: Reversing a queue using a stack: all elements dequeued (pushed onto stack), then re-enqueued by popping.

8.2. Algorithm 2 — Reverse In-Place (Two-Pointer)

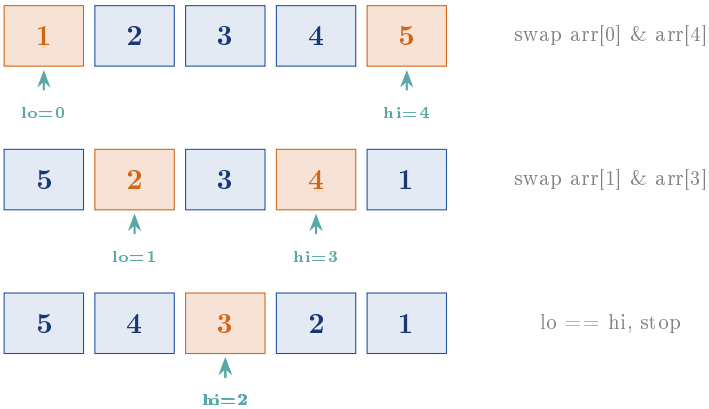


Figure 11: In-place reversal of `[1,2,3,4,5]`: orange cells are the current swap pair. Result: `[5,4,3,2,1]`.

8.3. Algorithm 3 — Left-Rotate by k Positions

$$[10, 20, 30, 40, 50] \xrightarrow{k=2} [30, 40, 50, 10, 20]$$

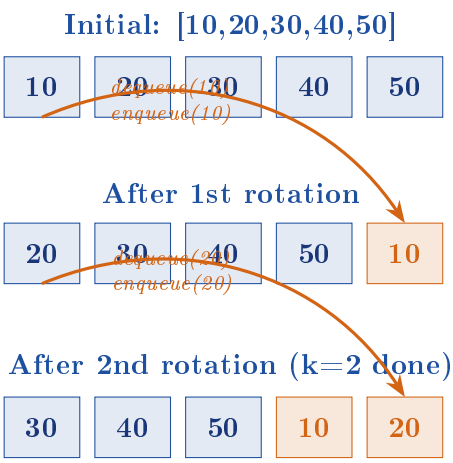


Figure 12: Left-rotation by 2: element 10 is dequeued and re-enqueued at the rear, then element 20 is.

9. Time and Space Complexity Analysis

9.1. Operation Complexities

Operation	Linear Array	Circular Array	Linked List
`enqueue(x)`	$O(1)$ amortised *($O(n)$ worst on shift)*	$\mathbf{O(1)}$ always	$\mathbf{O(1)}$ always
`dequeue()`	$O(1)$	$O(1)$	$O(1)$
`peek()`	$O(1)$	$O(1)$	$O(1)$
`isEmpty()`	$O(1)$	$O(1)$	$O(1)$
`isFull()`	$O(1)$	$O(1)$	N/A (unbounded)
`size()`	$O(1)$	$O(1)$	$O(1)$
`search(x)`	$O(n)$	$O(n)$	$O(n)$
`reverseQueue()`	$O(n)$ time, $O(n)$ space	$O(n)$ time, $O(n)$ space	$O(n)$ time, $O(n)$ space
`reverseInPlace()`	$O(n)$ time, $O(1)$ space	$O(n)$ time, $O(1)$ space	$O(n)$ time, $O(1)$ space
`rotateByK(k)`	$O(k)$	$O(k)$	$O(k)$
Space Complexity			
Total space	$O(n)$ fixed array	$O(n)$ fixed array	$O(n)$ heap nodes
Extra per element	None	None	1 pointer (`next`)
Max capacity	Fixed at creation	Fixed at creation	Heap limit

Table 1: Time and space complexities for all queue operations. n = number of elements currently in the queue.

9.2. Understanding Amortised $O(1)$

In the linear queue, `enqueue` is normally $O(1)$. Occasionally, it triggers a $O(n)$ shift. This shift only occurs after n cheap $O(1)$ operations. Averaged over all n operations, the cost is $O(n)/n = O(1)$ per call. This is **amortised** $O(1)$ — cheap on average but with occasional expensive peaks. The circular queue avoids this completely: every `enqueue` is $O(1)$ in the worst case.

9.3. When to Choose Which Implementation

Situation	Best Choice
Max size known, performance-critical	Circular array queue
Max size unknown / highly variable	Linked-list queue
Memory-constrained (no pointer overhead)	Circular array
Guaranteed $O(1)$ worst-case	Circular array or linked-list
Both-end insertion and deletion	`std::deque` (STL)
Priority-based ordering	`std::priority_queue` (STL)

10. Common Mistakes Cheat-Sheet

1. **Off-by-one after compaction:** After shifting n elements to $0 \ldots n-1$, set `rear = n-1`. NOT `n` or `n+1`.

2. **Wrong upper bound in `reverseUpToK`:** Use `hi = front + k - 1`. Never write `(k-1) - front`.

3. **Circular queue display with `while(temp <= rear)`:** Fails when `front > rear` (wrapped state). Use `for(int i=0; i<count; i++)` with `idx=(idx+1)%capacity`.

4. **No destructor in class with `new`:** Always write a destructor calling `delete` or `delete[]`.

5. **Not resetting `rear_` to `nullptr` after last dequeue:** Leaves a dangling pointer that causes heap corruption on the next enqueue.

6. **Missing `return` statement in factory function:** A non-void function with no return is undefined behaviour in C++.

7. **`count` not incremented in first-insertion branch:** If the empty-queue branch returns before `count++`, all subsequent operations have an off-by-one element count.

11. Summary and Quick Reference

Everything You Need to Remember — Queue Data Structure

1. A queue is **FIFO**: the oldest element always leaves first.
2. Two pointers: `front` (exit/oldest) and `rear` (entry/newest).
3. **Enqueue** ⇒ rear advances. **Dequeue** ⇒ front advances.
4. **False overflow** = rear hits the end but front > 0. The circular queue solves this permanently.
5. **Circular trick**: `rear = (rear+1) % capacity`. No shifting, $O(1)$ always.
6. **Circular display**: loop with `count` as bound, not `rear`.
7. **Linked-list**: no false overflow, no fixed size. Extra pointer per node.
8. **Always** add a destructor when your class uses `new`.
9. **Reversal**: stack-based ($O(n)$ space) or two-pointer swap ($O(1)$ space, array only).
10. **Rotation by** k: dequeue front, enqueue it at rear, repeat k times.

11.1. Implementation Comparison Cheat-Sheet

Feature	Linear Array	Circular Array	Linked List
False overflow possible	**Yes**	**No**	**No**
$O(1)$ enqueue (worst)	**No**	**Yes**	**Yes**
Fixed capacity	Yes	Yes	No
Memory per element	1 int	1 int	1 int + 1 pointer
Cache efficiency	Best	Best	Lower
Implementation simplicity	Low	Medium	Medium

11.2. Classic Queue-Based Interview Problems

1. **BFS on a Graph/Tree** — Queue to explore nodes level by level.
2. **Level-Order Traversal of Binary Tree** — Dequeue node, print, enqueue children.
3. **First Non-Repeating Character in a Stream** — Queue of unique characters.
4. **Sliding Window Maximum** — Deque for $O(1)$ amortised per window step.
5. **Implement Stack Using Two Queues** — Classic LIFO-from-FIFO exercise.
6. **Implement Queue Using Two Stacks** — Classic FIFO-from-LIFO exercise.

7. **Generate Binary Numbers 1 to n** — Enqueue "1"; dequeue, print, enqueue with "0"/"1" appended.
8. **Rotten Oranges (Multi-Source BFS)** — All rotten oranges enqueued initially.
9. **Number of Islands** — BFS flood-fill using a queue.
10. **Task Scheduler** — Priority queue to greedily schedule CPU tasks.

End of Notes

Queue Data Structure in C++ — Complete Beginner-to-Advanced Reference

All implementations are corrected, fully memory-safe, and production-quality.

Every concept introduced from scratch with step-by-step diagrams and explanations.